- Identify and justify the physical quantities to be measured.

- Select propulsion and rudder motors, justify the choice with datasheets, and compare alternatives.

- Choose appropriate sensors and explain the interface used (analog, digital, I2C, ...).

- Describe and justify the power stage (H-bridge, PWM control, compatibility with motor and supply).

- Choose and size the battery (voltage, capacity, autonomy).

-Propose hardware protection.

EEE Components

- **MOTOR OF THE BOAT (** https://www.amazon.fr/DC MOTOR **)**

Checking the motors

- A **70cm boat** is relatively small, this suggests it **doesn't need a very high-power motor**.
- 12V motors are typically **smaller, lighter, and more suitable** for compact boats like yours.
- 24V motors are more powerful, but usually **overkill** for a boat under 1 meter unless we are trying to go **very fast** or carry a **heavy load**. But not the case

- **Size + mounting**: The physical size, shape, terminal positions matter.

specifications;

Marque;	Tnfeeon
Nom de modèle;	Tnfeeonz43xkur0hs
Tension;	12 Volts
Puissance (chevaux);	100 Watt
Matériau;	Cuivre, Métal

- **Poids :** 350g
- **Vitesse de rotation ;** 12000 tr/min
- **Niveau de bruit inférieur;** 42 dB
- **courant de démarrage ;** plus de 10 A
- price; **14.67€**
- Dimensions or Sizing;



Estimate Motor Current

Power (Watts) = Voltage (V)×Current (A) = IV
⇒Current (A) = Voltage (V) / Power (W) = 100/12 = 8.333A

motor choosed; 12 V 775 Motor (~12000 rpm)

- **No-load behavior**

The motor might spin ~12,000 rpm with no load (as rated). But once the prop is attached and pushing water, rpm will drop.

Angular speed : N=9500, $\omega = 2\pi N/60 = 995$ rad/s

Calculate Output Torque

$P = T \cdot \omega$, $T = P/\omega$
T= 100/995= 0.101N.m

Electrical input power and Current:

At 100 W mechanical output, the motor draws more than 100 W electrically (due to losses).
If efficiency η were 80% (a typical value for 775 DC motors):

$\eta = P_{out}/P_{in} = 0.8$

$P_{in} = V \cdot I$

Efficiency for DC 775 motors is typically ~75–85%.
Assume 80 % efficiency:

Pin= 100/0.8=125 W

I=Pin/V= 125/12= 10.4A

Speed drop & margin
In practice, maybe your rpm will drop to ~8,000–10,000 rpm with load depending on drag, efficiency, prop slip, etc. So having a motor with no-load >/=12,000 rpm gives a safty margin.

- **BATTERY SYSTEM (** https://www.amazon.fr/NASTIMA-Rechargeable-Protection-poussettes-Camping-Cars/dp/B0B6P4DD3J?th=1 **)**
- **Voltage**: We need 12 V (since we decided on a 12 V motor).

- **Capacity (Ah)**: How many amp-hours we need depends on how much current your motor draws, and how long we want to run. E.g. if motor draws 5 A, and we want 1 hour, we need at least ~5 Ah (ignoring losses).
- **Weight :**

- **Size + mounting**: The physical size, shape, terminal positions matter.

Characteristics;

- **Nombre de batteries ;** 1 Lithium-polymère nécessite des piles.
- Capacité : 10 000 mAh (10 AH) | Tension nominale : 12 V
- Dimensions : 120 x 65 x 31 mm
- Poids : 494 g
- Connexions électriques : sortie câble rouge/noir 1,5 mm2 - Entrée : jack femelle 5,5 x 2,1 mm

- Price; **72,99€**

Autonomy Calculations

Autonomy (hours) = Motor Current Draw (A) / Battery Capacity (Ah)
= 10/10 = 1h = 60min

- **Arduino microcontroller board (https://www.amazon.Arduino uno R4-WIFI)**

https://store-UNOwifi



Characteristics ;

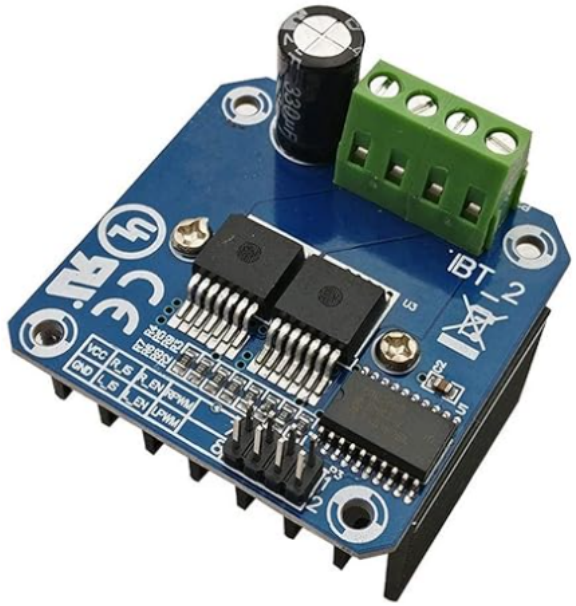
- **Marque;** ARDUINO
- **Nom de modèle;** UNO R4
- **Technologie de connectivité;** USB
- **Composants inclus ;** Carte Wi-Fi UNO R4, matrice LED intégrée 12 x 8, connecteur Qwiic et appareils compatibles avec interfaces UART, SPI, I2C ou CAN
- **Norme de communication sans fil ;** Bluetooth
- Price ; **30.50€**
-

- **H-BRIDGE (** https://www.amazon.fr/Hailege-Refroidissement-Semi-conducteur-intelligente-Entra%C3%AEnement/dp/B07Y4TTK3Z **)**

The L298N motor driver board is a high-power, dual H-bridge motor driver that can be used to control two DC motors or one stepper motor. It is a popular choice for robotics projects, DIY projects, and other applications that require precise motor control.

The L298N board is easy to use and can be controlled by a variety of microcontrollers, including the Arduino. It has a built-in 5V regulator, so it can be powered from a single supply voltage. The board also has a number of protection features, including overcurrent protection and thermal protection.

Note: Pour éviter d'endommager la tension puce de stabilisation, s'il vous plaît utiliser une logique alimentation 5V externe lors de l'utilisation de plus de 12V tension de commande



So we want:

- **Voltage range**: 5–15 V (safely covers $LiFePO_4$ voltage)
- **Continuous current**: ≥ 15 A
- **Peak current**: ≥ 30–40 A (safety margin)
- **PWM + direction inputs** compatible with Arduino, ESP32, Raspberry Pi, etc.

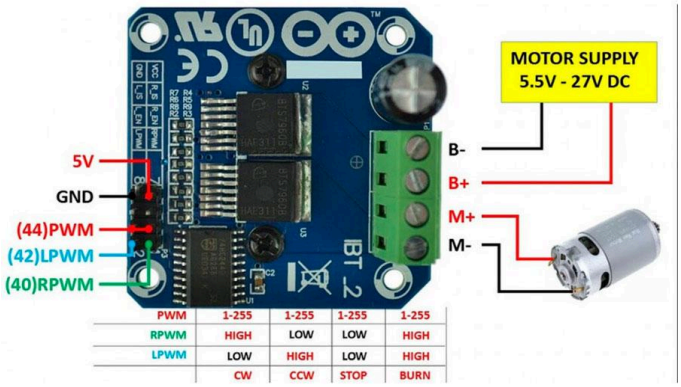
BTS7960 43 A Dual H-Bridge Driver

- **Specs:** 5–27 V, 43 A peak, logic-level inputs
- **Ideal for:** 12 V 775 motor class
- **Advantages:**

    - Built-in current sensing & over-temperature protection
    - Handles motor direction and PWM speed control
    - Simple wiring (IN1/IN2/PWM)
    - Widely available and inexpensive (~10 €–15 €)
- **Cooling:** Small heatsink included — add airflow for long high-load runs

Battery + → H-bridge VCC
Battery – → H-bridge GND
Motor +/– → H-bridge OUT1/OUT2
Controller (Arduino etc.) PWM & direction pins → H-bridge inputs

Fuse ; a fuse (~15–20 A) between the battery and H-bridge for protection.
wires ; 12–14 AWG (2–3 mm²) to minimize losses



- CURRENT SENSOR (https://www.amazon.fr/HiLetgo-ACS758LCB-050B-PFF-T-lin%C3%A9aire-ACS758LCB-passante/dp/B0832PHKB5)

    Battery + → Current Sensor IN+

Sensor OUT+ → H-Bridge VCC+
Battery – → H-Bridge GND (common)
Motor connected to H-Bridge outputs
Sensor Vcc (5 V) → Arduino 5 V
Sensor GND → Arduino GND
Sensor OUT (analog) → Arduino analog pin (e.g., A0)

For ACS758-050B,

You'll read an analog voltage representing current:

I= (Vout−Vzero) / Sensitivity

- Vzero ≈ 2.5V (no current)

- Sensitivity ≈ 40 mV/A
  So if you read 3.3 V → (3.3–2.5)/0.04=20A

```
        Arduino;
float sensorValue = analogRead(A0);
float voltage = sensorValue * (5.0 / 1023.0);
float current = (voltage - 2.5) / 0.04; // for ACS758-050B
if (current > 25.0) {
  // e.g., cut power, alarm, or log event
}
```

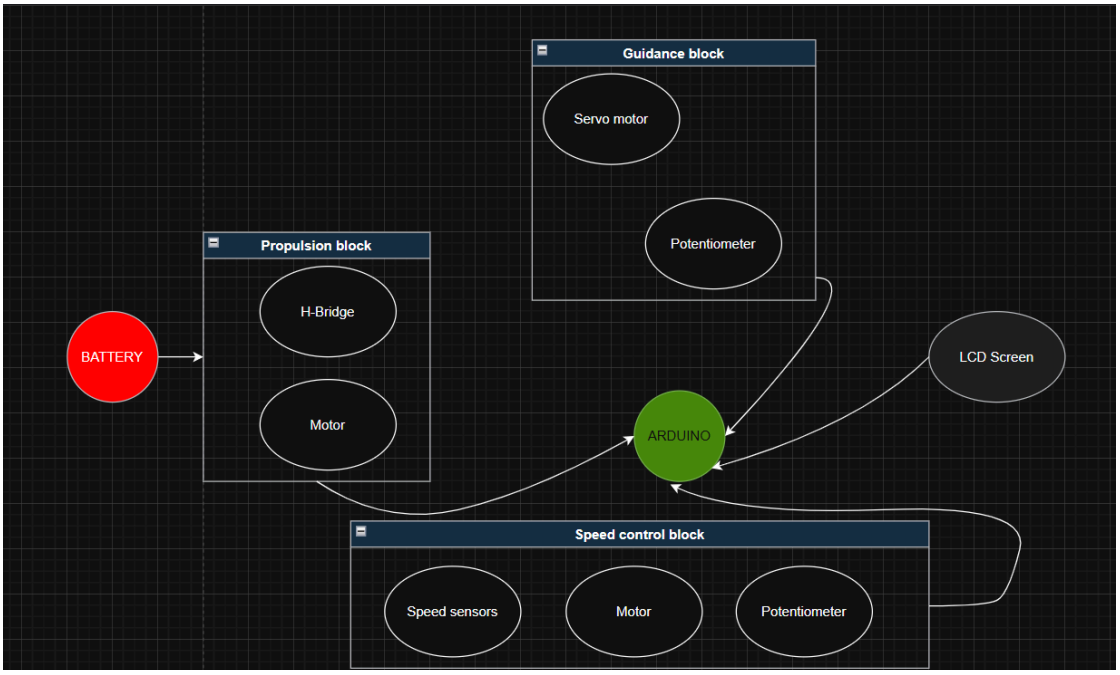
Characteristics;

- VCC: 5V DC Nominal
- Measurement Range: -5 to +5 A
- Dimensions: 31.73 mm 13.25 mm 14 mm
- Price; **2,95€**
  https://www.moussasoft.com/acs712-capteur-de-courant-arduino/

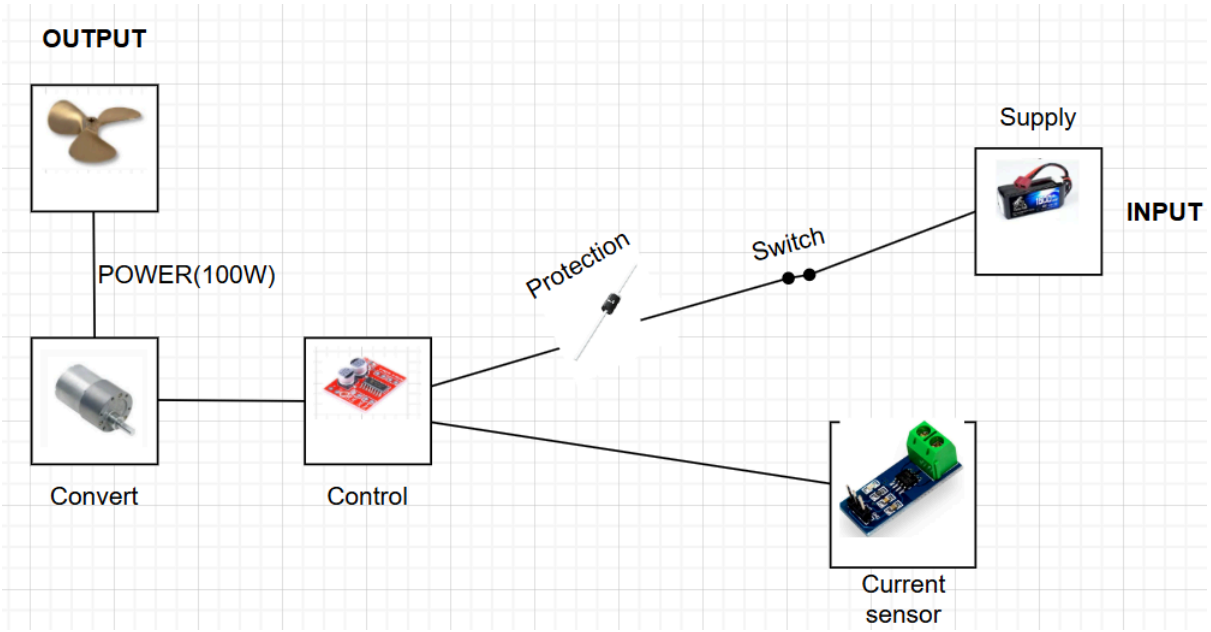
- DIODE AND SWITCH"
  - A diode for protection of our circuit,It allows current to flow easily in one direction but severely restricts current from flowing in the opposite direction.
  - Switch for controlling the opening and closing of our electric circuit.It allows you to control the flow of current in a circuit (without having to

manually cut or splice wires).

Functional block



SCHEME



MIA

- SERVO MOTOR

https://www.amazon.fr/SERVOMOTOR

- Speed sensors
  https://www.speedsensor

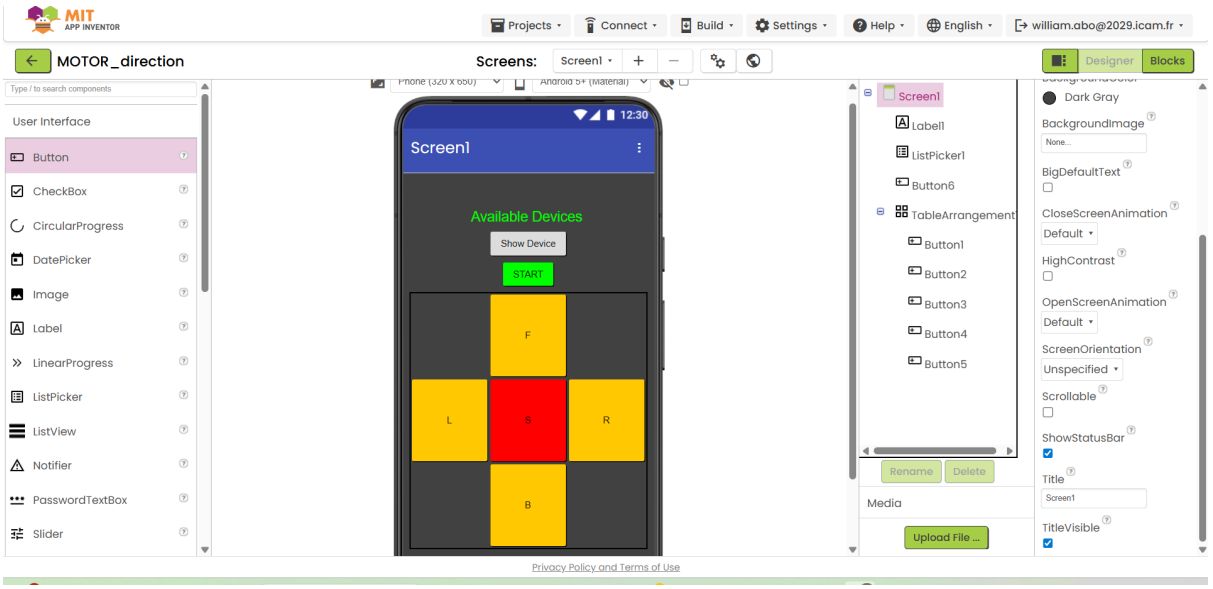
- Electronic compass(Magnetometer)

  https://www.amazon.GY-273-Magnetometer

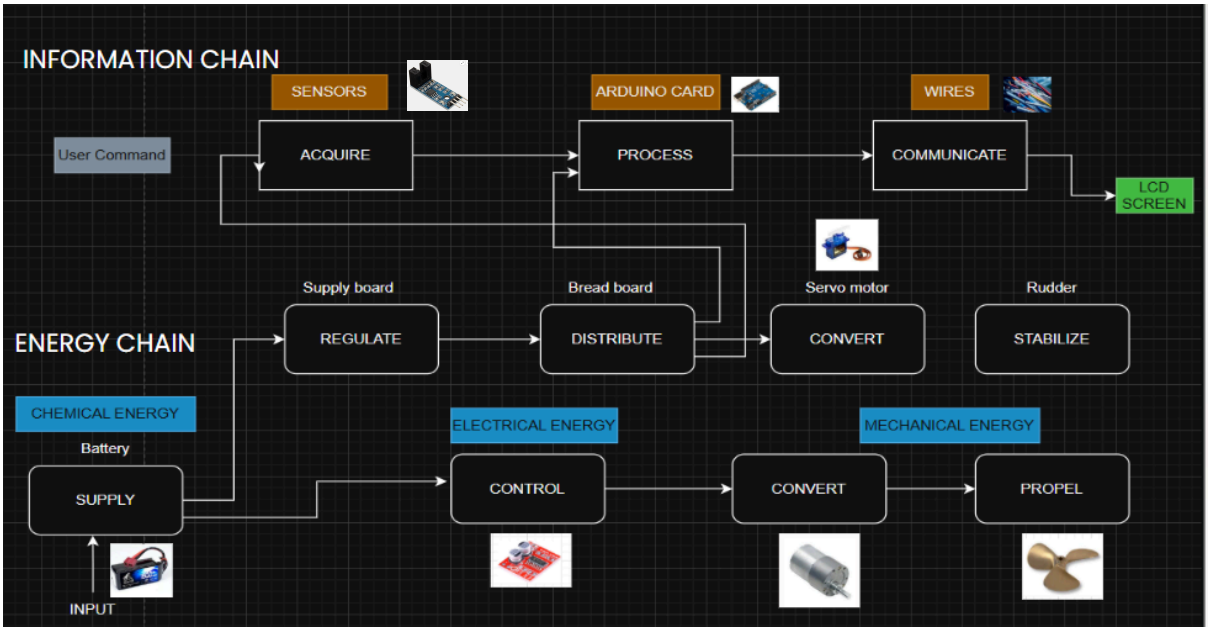
- Software app inventer
  (https://fr.scribd.com/document/861895763/Tutoriel-AppInventor-Bluetooth)

  https://www.youtube.com/watch?v=FIPOh50Zub0

  We proceed in Four parts for the application
    - Conception of the screen
    - Ouvrir App inventor et créer un projet
    - Ajouter des composant d'interface utilisateur
    - Block programming (Logic of the app)
    - Accéder à l'éditeur blocs pour programmer le comportement de l'application pour le projet
    - Initialisation de la connexion bluetooth
    - Compiler l'application une fois la programmation en bloc terminer
    - Arduino programming
    - Programmation sur L'IDE Arduino
    - Test application and connect to phone



GLOBAL INFORMATION AND ENERGY CHAIN



Échanger des données avec un appareil IoT via Wi-Fi

L'ESP sert de serveur web ou de point d'accès.

L'application App Inventor utilise le composant Web pour envoyer des requêtes HTTP à l'ESP (ex: `http://192.168.4.1/led=on`).

Code ESP32 (Arduino IDE)

Le bateau agit comme serveur web Wi-Fi.
Quand tu appuies sur un bouton dans App Inventor, il envoie une requête HTTP à l'ESP (par exemple `http://192.168.4.1/avant`).

Code speed, current sensor and motor

```
// Définition des broches
#define POTENTIOMETER_PIN A0 // Potentiomètre
#define CURRENT_SENSOR_PIN A1 // Capteur ACS712
#define ENA 4                 // Broche PWM pour contrôle de vitesse
#define IN1 3                 // Direction du moteur
#define IN2 11                // Direction du moteur
#define SPEED_SENSOR_PIN 12   // Capteur infrarouge

// Variables pour le capteur ACS712
const float ACS712_SENSITIVITY = 0.185; // Sensibilité pour 5A
const int ACS712_OFFSET = 512;          // Valeur moyenne à 0A
```

```
// Variables pour le capteur de vitesse
volatile unsigned int pulseCount = 0;   // Nombre d'impulsions
float motorRPM = 0.0;                   // RPM du moteur

void setup() {
  // Configuration des broches
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(SPEED_SENSOR_PIN, INPUT_PULLUP);

  // Configuration de l'interruption pour le capteur de vitesse
  attachInterrupt(digitalPinToInterrupt(SPEED_SENSOR_PIN), countPulse, FALLING);

  // Initialisation du moteur
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 0);
}

void loop() {
  // Lecture du potentiomètre et contrôle du moteur
  int potValue = analogRead(POTENTIOMETER_PIN);       // Lecture du potentiomètre (0 à 1023)
  int motorSpeed = map(potValue, 0, 1023, 0, 255);    // Conversion en 0-255
  analogWrite(ENA, motorSpeed);

  // Définir la direction du moteur
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  // Calcul de la vitesse du moteur
  motorRPM = calculateRPM();

  // Lecture de l'intensité du moteur
  float current = readCurrent();

  // (LCD supprimé — aucune sortie visuelle ici)
  // Vous pouvez ajouter Serial.print() si vous voulez surveiller via le moniteur série

  delay(500); // Pause pour actualisation
}

// Fonction pour compter les impulsions du capteur de vitesse
void countPulse() {
  pulseCount++;
}

// Fonction pour calculer la vitesse en RPM
float calculateRPM() {
  static unsigned long lastTime = 0;
  unsigned long currentTime = millis();
```

```
  unsigned int pulses;

  // Calculer le temps écoulé et les impulsions détectées
  noInterrupts();
  pulses = pulseCount;
  pulseCount = 0;
  interrupts();

  unsigned long timeElapsed = currentTime - lastTime;
  lastTime = currentTime;

  if (timeElapsed > 0) {
    return (pulses / 20.0) * (60000.0 / timeElapsed); // Conversion en RPM
  } else {
    return 0.0; // Si aucune impulsion détectée
  }
}

// Fonction pour lire l'intensité avec le capteur ACS712
float readCurrent() {
  int sensorValue = analogRead(CURRENT_SENSOR_PIN);
  return (sensorValue - ACS712_OFFSET) * (5.0 / 1024.0) / ACS712_SENSITIVITY;
}
```

Code for Servo motor

```
#include <Servo.h>

Servo direction;       // Objet servo
const int servoPin = 9;      // Pin signal du servo
const int potPin = A0;       // Pin analogique du potentiomètre

void setup() {
  direction.attach(servoPin);
  Serial.begin(9600);
}

void loop() {
  int potValue = analogRead(potPin);              // Lire la valeur du potentiomètre (0-1023)
  int angle = map(potValue, 0, 1023, 0, 180);     // Convertir en angle servo (0-180)

  direction.write(angle);                         // Positionner le servo
  Serial.print("Potentiomètre: ");
  Serial.print(potValue);
  Serial.print(" -> Angle servo: ");
  Serial.println(angle);

  delay(15);  // Temps pour que le servo atteigne la position
}
```

IDE MODULE WIFI

```
#include <WiFiNINA.h>
#include <Servo.h>

// --- Configuration Wi-Fi ---
const char* ssid = "Bateau";
const char* password = "12345678";

// --- Pins moteur H-bridge ---
const int IN1 = 5;
const int IN2 = 4;
const int ENA = 6;  // PWM pour vitesse moteur

// --- Pin servo ---
const int servoPin = 9;

Servo direction;
WiFiServer server(80);

void setup() {
  Serial.begin(9600);
```

```
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);

  // Au démarrage moteur à l'arrêt
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 0);

  direction.attach(servoPin);
  direction.write(90); // servo centré

  Serial.print("Connexion Wi-Fi...");
  int status = WL_IDLE_STATUS;
  while (status != WL_CONNECTED) {
    status = WiFi.begin(ssid, password);
    delay(2000);
    Serial.print(".");
  }
  Serial.println("\nConnecté !");
  Serial.print("IP locale: ");
  Serial.println(WiFi.localIP());

  server.begin();
}

void loop() {
  WiFiClient client = server.available();
  if (client) {
    String requete = client.readStringUntil('\r');
    client.flush();

    if (requete.indexOf("/avant") != -1) {
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, LOW);
      analogWrite(ENA, 255); // vitesse max
      Serial.println("AVANT");
    }
    else if (requete.indexOf("/stop") != -1) {
      analogWrite(ENA, 0);  // arrêt moteur
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
```

```
      Serial.println("STOP");
    }
    else if (requete.indexOf("/gauche") != -1) {
      direction.write(45);
      Serial.println("GAUCHE");
    }
    else if (requete.indexOf("/droite") != -1) {
      direction.write(135);
      Serial.println("DROITE");
    }
    else if (requete.indexOf("/centre") != -1) {
      direction.write(90);
      Serial.println("CENTRE");
    }

    // Réponse HTTP simple
    client.println("HTTP/1.1 200 OK");
    client.println("Content-Type: text/html");
    client.println();
    client.println("<h1>Commande reçue !</h1>");
    client.stop();
  }
}
```

Tu peux contrôler la **vitesse** en remplaçant :
analogWrite(moteurENA, 255);

Quand l'ESP32 démarre, il crée son propre Wi-Fi nommé `Bateau`.
Ton téléphone s'y connecte, puis ton appli envoie des commandes à l'adresse :
http://192.168.4.1/avant
http://192.168.4.1/stop
http://192.168.4.1/gauche
http://192.168.4.1/droite

Test

1. Allume ton ESP32 → il crée le Wi-Fi `Bateau`.

2. Connecte ton téléphone à ce réseau.

3. Lance ton appli MIT App Inventor.

4. Appuie sur les boutons → le bateau bouge

Note: Si tu veux que ton Arduino crée son propre réseau Wi-Fi, ce code ne suffit pas : le UNO WiFi Rev2 ne peut pas faire de "softAP" (point d'accès). Seules les cartes ESP32 / ESP8266 le peuvent.

Electrical Input Power:
$P_{in} = V \cdot I$
Mechanical Output Power:
$P_{out} = T \cdot \omega$
Losses:
$P_{loss} = I^2.R + P_{friction} + P_{windage}$
Efficiency:
$\eta = P_{out} / P_{in}$

```
volatile uint32_t pulseCount = 0;

void IR_interrupt() {
  pulseCount++;
}

void setup() {
  pinMode(IR_OUT_PIN, INPUT);
  attachInterrupt(digitalPinToInterrupt(IR_OUT_PIN), IR_interrupt, RISING);

  Wire.begin();
  magnetometer.begin();  // e.g. HMC5883L init
}

void loop() {
  // Compute RPM
  uint32_t count = pulseCount;
  pulseCount = 0;
  float rpm = (count * 60.0) / measurementIntervalSeconds;
```

```
  // Read magnetometer
  Vector mag = magnetometer.read();  // returns x, y, z
  float heading = atan2(mag.y, mag.x) * 180.0 / PI;  // in degrees

  // Optionally correct heading by tilt using accelerometer if you have one

  // Print or use values
  Serial.print("RPM: "); Serial.print(rpm);
  Serial.print(" | Heading: "); Serial.println(heading);

  delay(measurementIntervalMs);
}
```

FINAL

```
#include <WiFiNINA.h>
#include <Wire.h>
#include <Servo.h>
#include <QMC5883LCompass.h>

// ==================== Wi-Fi Configuration ====================
const char* ssid = "Bateau";
const char* password = "12345678";

WiFiServer server(80);

// ==================== Motor Control (BTS7960) ====================
#define RPWM 5
#define LPWM 6
#define R_EN 7
#define L_EN 8

// ==================== Sensors & Servo ====================
#define CURRENT_PIN A0
#define IR_SENSOR_PIN 2
#define SERVO_PIN 9

Servo direction;
QMC5883LCompass compass;
```

```
// ==================== Global Variables ====================
volatile unsigned long lastPulseTime = 0;
volatile unsigned long pulseInterval = 0;
float rpm = 0;

float currentOffset = 2.5;  // 0A = ~2.5V (ACS758)
float sensitivity = 0.04;   // 40 mV/A for ACS758-50B

// ==================== Setup ====================
void setup() {
  Serial.begin(9600);
  Wire.begin();

  // H-Bridge setup
  pinMode(RPWM, OUTPUT);
  pinMode(LPWM, OUTPUT);
  pinMode(R_EN, OUTPUT);
  pinMode(L_EN, OUTPUT);
  digitalWrite(R_EN, HIGH);
  digitalWrite(L_EN, HIGH);

  // Servo setup
  direction.attach(SERVO_PIN);
  direction.write(90); // Center rudder

  // IR sensor for RPM
  pinMode(IR_SENSOR_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(IR_SENSOR_PIN), countPulse, FALLING);

  // Compass setup
  compass.init();
  compass.setCalibration(-635, 1243, -2943, 1184, -1115, 2087);

  // Wi-Fi connection
  Serial.print("Connecting to Wi-Fi...");
  int status = WL_IDLE_STATUS;
  while (status != WL_CONNECTED) {
    status = WiFi.begin(ssid, password);
    delay(2000);
    Serial.print(".");
```

```
  }
  Serial.println("\nConnected!");
  Serial.print("Local IP: ");
  Serial.println(WiFi.localIP());

  server.begin();
  Serial.println("HTTP server started");
}

// ==================== Main Loop ====================
void loop() {
  WiFiClient client = server.available();
  if (client) {
    String request = client.readStringUntil('\r');
    client.flush();

    handleCommand(request);

    // Send simple response
    client.println("HTTP/1.1 200 OK");
    client.println("Content-Type: text/html");
    client.println();
    client.println("<h1>Command received!</h1>");
    client.stop();
  }

  // Read and display telemetry
  readTelemetry();
  delay(500);
}

// ==================== Functions ====================

// Handle incoming Wi-Fi commands
void handleCommand(String req) {
  req.toLowerCase();

  if (req.indexOf("/avant") != -1) {
    controlMotor(180, true);
    Serial.println("FORWARD");
```

```
  }
  else if (req.indexOf("/arriere") != -1) {
    controlMotor(180, false);
    Serial.println("REVERSE");
  }
  else if (req.indexOf("/stop") != -1) {
    controlMotor(0, true);
    Serial.println("STOP");
  }
  else if (req.indexOf("/gauche") != -1) {
    direction.write(45);
    Serial.println("LEFT");
  }
  else if (req.indexOf("/droite") != -1) {
    direction.write(135);
    Serial.println("RIGHT");
  }
  else if (req.indexOf("/centre") != -1) {
    direction.write(90);
    Serial.println("CENTER");
  }
}

// Motor control via BTS7960
void controlMotor(int pwmValue, bool forward) {
  if (forward) {
    analogWrite(RPWM, pwmValue);
    analogWrite(LPWM, 0);
  } else {
    analogWrite(RPWM, 0);
    analogWrite(LPWM, pwmValue);
  }
}

// Telemetry: Heading, RPM, Current
void readTelemetry() {
  compass.read();
  int heading = compass.getAzimuth();

  float sensorVoltage = analogRead(CURRENT_PIN) * (5.0 / 1023.0);
```

```
  float motorCurrent = (sensorVoltage - currentOffset) / sensitivity;
  rpm = calculateRPM();

  Serial.print("Heading: "); Serial.print(heading);
  Serial.print("° | RPM: "); Serial.print(rpm);
  Serial.print(" | Current: "); Serial.print(motorCurrent, 2);
  Serial.println(" A");
}

// IR speed sensor interrupt
void countPulse() {
  unsigned long now = micros();
  pulseInterval = now - lastPulseTime;
  lastPulseTime = now;
}

// RPM calculation
float calculateRPM() {
  if (pulseInterval > 0) {
    float freq = 1000000.0 / pulseInterval;
    return (freq * 60.0); // 1 pulse per revolution
  } else {
    return 0;
  }
}
```

CODE MOTOR AND HBRIDGE WITH ESP32

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "TON_WIFI";
const char* password = "MOT_DE_PASSE";

WebServer server(80);

// Pins H-Bridge
#define IN1 26
#define IN2 27
#define ENA 25
```

```
int vitesse = 0;
int pwmChannel = 0;
int pwmFreq = 20000;
int pwmResolution = 8;

// ================ HTML ================
const char pageHTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Controle Moteur DC</title>
<style>
body {
  font-family: Arial;
  background: #111;
  color: white;
  text-align: center;
}
button {
  padding: 15px 25px;
  margin: 10px;
  font-size: 18px;
  border: none;
  border-radius: 10px;
  cursor: pointer;
}
#avant { background: #00c853; }
#arriere { background: #2979ff; }
#stop { background: #d50000; }

input[type=range] {
  width: 80%;
}
</style>
</head>

<body>
<h1>ESP32 - Moteur DC</h1>
```

```
<button id="avant" onclick="cmd('avant')">AVANT</button>
<button id="arriere" onclick="cmd('arriere')">ARRIÈRE</button>
<button id="stop" onclick="cmd('stop')">STOP</button>

<h2>Vitesse: <span id="val">0</span></h2>
<input type="range" min="0" max="255" value="0" oninput="vitesse(this.value)">

<script>
function cmd(c) {
  fetch('/' + c);
}

function vitesse(v) {
  document.getElementById("val").innerText = v;
  fetch('/vitesse?val=' + v);
}
</script>

</body>
</html>
)rawliteral";
// ========================================

void handleRoot() {
  server.send(200, "text/html", pageHTML);
}

void moteurAvant() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  server.send(200, "text/plain", "AVANT");
}

void moteurArriere() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  server.send(200, "text/plain", "ARRIERE");
}
```

```
void moteurStop() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  server.send(200, "text/plain", "STOP");
}

void setVitesse() {
  if (server.hasArg("val")) {
    vitesse = server.arg("val").toInt();
    ledcWrite(pwmChannel, vitesse);
  }
  server.send(200, "text/plain", "OK");
}

void setup() {
  Serial.begin(115200);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);

  ledcSetup(pwmChannel, pwmFreq, pwmResolution);
  ledcAttachPin(ENA, pwmChannel);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nConnecté !");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/avant", moteurAvant);
  server.on("/arriere", moteurArriere);
  server.on("/stop", moteurStop);
  server.on("/vitesse", setVitesse);

  server.begin();
}
```

```
void loop() {
  server.handleClient();
}
```